```
WebApplication app = builder.Build();

app.MapGet("/register/{username}", RegisterUser);

app.Run();
```

❶ The extension method registers all the services associated with the EmailSender.

So far, you've seen how to register the simple DI cases in which you have a single implementation of a service. In some scenarios, you may have multiple implementations of an interface. In section 9.3 you'll see how to register these with the container to match your requirements.

9.3 Registering a service in the container multiple times

One advantage of coding to interfaces is that you can create multiple implementations of a service. Suppose that you want to create a more generalized version of `IEmailSender` so that you can send messages via Short Message Service (SMS) or Facebook, as well as by email. You create the interface for it as follows,

```
public interface IMessageSender
{
    public void SendMessage(string message);
}
```

as well as several implementations: `EmailSender`, `SmsSender`, and `FacebookSender`. But how do you register these implementations in the container? And how can you inject these implementations into your `RegisterUser` handler? The answers vary slightly, depending on whether you want to use all the implementations in your consumer or only one.